



NAMAL COMPLAINT MANAGEMENT SYSTEM

Database Design Document

V 2.0

By

1	Sundeeep Kumar	NUM-BSCS-2024-75
2	Rehan Tariq	NUM-BSCS-2024-66
3	Aimen Shafiq	NUM-BSCS-2024-04

Department of Computer Sciences

Namal University

Mianwali, Pakistan

Submission Date: 14th June, 2026

REVISION HISTORY

Date	Version	Description	Approved by
14/06/26	V 2.0	The objective of this project is to design relational schema models in 3NF from ERD, database implementation using DDL and DML programming code, and designing an easy-to-use graphical user interface for manipulating the database. The graphical user interface is embedded with the database using stored procedures, functions, and triggers to ensure secure and efficient data manipulation.	
15/05/26	V 1.0	Based on the feedback received on the previous document, several improvements were made to enhance the overall quality of the project. The ERD and relational schema were refined, business rules were updated for better clarity, and minor revisions were made to the database design and documentation structure. Additionally, formatting and content organization were improved to ensure consistency and completeness.	

Contents

Revision History	1
1 PROJECT OVERVIEW	3
1.1 INTRODUCTION	3
1.2 PROBLEM STATEMENT	3
1.3 PROJECT OBJECTIVES	3
2 CONCEPTUAL DATABASE DESIGN	4
2.1 ENTITY	4
2.2 DATA DICTIONARY	5
2.3 BUSINESS RULES	10
2.4 ENTITY RELATIONSHIP DIAGRAM	10
3 LOGICAL DATABASE DESIGN	12
3.1 RELATIONAL SCHEMA	12
3.2 FUNCTIONAL DEPENDENCIES	12
3.3 NORMALIZATION	13
3.3.1 Partial Dependencies	13
3.3.2 Transitive Dependencies	13
4 IMPLEMENTATION	17
4.1 LANGUAGE/Framework	17
4.2 DATABASE CONNECTIVITY	17
4.3 STORED PROCEDURES AND FUNCTIONS	17
4.4 INTERFACES	18
References	21
Appendix: AI Usage and Prompts	22

PROJECT OVERVIEW

1.1. INTRODUCTION

The Namal Complaint Management System is built specifically for managing complaints that relate to maintenance problems at Namal University. The system offers an integrated forum on which both the students, staff and faculty can file complaints about any kind of problem, ranging from electricity, plumbing, internet connection, damaged furniture, housekeeping, among others. Through the system, it becomes easy for maintenance personnel and management to handle, allocate, and solve these problems. They will also be able to follow up with the progress of the solution of these complaints.

1.2. PROBLEM STATEMENT

In terms of handling complaints, Namal University still relies on manual methods of communication like personal complaints, calls, or text messages, causing delays and communication issues. There is no established method that would allow complaints regarding maintenance issues such as electricity failures, plumbing issues, internet problems, or other maintenance concerns to be recorded, tracked, and followed-up. In this way, the lack of proper recording and monitoring systems causes potential problems with complaints being forgotten, delayed, or even submitted repeatedly. Such inefficiency causes problems for both students and maintenance workers, making it hard for them to resolve the issues efficiently and in a timely manner.

1.3. PROJECT OBJECTIVES

- The platform would serve as a friendly portal that allows the students and staff to lodge their complaints regarding electrical works, plumbing, internet, and other types of complaints.
- This system should facilitate the automation of the registration of complaints to minimize the workload of manually handling such tasks.
- It is essential to ensure the availability of real-time information about each complaint, which can have any of the following statuses – pending, processing, and resolved.
- This system should ensure the appropriate assignment of complaints to various departments for proper handling.
- A database is required to store records of all complaints made through the portal.

GITHUB REPOSITORY LINK:

https://github.com/SundeepKumar07/NAMAL_COMPLAINT_PORTAL

CONCEPTUAL DATABASE DESIGN

2.1. ENTITY

Sr. No	Entity Name	Description
1	User	Stores basic information of all system users including students, staff, and admin.
2	EndUser	Represents users who submit complaints such as students or university staff.
3	MaintenanceStaff	Stores information of staff responsible for resolving complaints.
4	Admin	Represents system administrators managing the system operations.
5	Department	Represents university departments linked with users and complaints.
6	Category	Defines types of complaints like electrical, plumbing, internet issues.
7	PriorityLevel	Defines urgency level of complaints and response time.
8	ComplaintStatus	Tracks current status of complaints (Pending, In Progress, Resolved).
9	Building	Represents university buildings where issues occur.
10	Location	Specifies exact room or floor of complaint occurrence.
11	Complaint	Main entity storing all complaint details submitted by users.
12	ComplaintAssignment	Represents assignment of complaints to maintenance staff.
13	WorkLog	Records maintenance work done on complaints.
14	Resolution	Stores final resolution details of complaints.
15	ComplaintImage	Stores images or attachments related to complaints.
16	Comment	Handles communication between users and staff on complaints.
17	Notification	Stores system alerts and updates sent to users.
18	NotificationTemplate	Predefined formats for sending notifications.
19	StatusHistory	Tracks history of complaint status changes.
20	Feedback	Stores user feedback and ratings after resolution.
21	AuditLog	Records system activities for monitoring and security.
22	StaffSpecialization	Shows staff expertise in complaint categories.

2.2. DATA DICTIONARY

2.2.1. USER

Sr. No	Name	Data Type	Constraint	Description
1	user_id	INT	PK, AUTO_INCREMENT	Unique user id
2	full_name	VARCHAR(100)	NOT NULL	User full name
3	email	VARCHAR(100)	UNIQUE, NOT NULL	User email
4	password	VARCHAR(255)	NOT NULL	Login password
5	phone_no	VARCHAR(20)	NULL	Contact number
6	account_status	VARCHAR(20)	NOT NULL	Active/Inactive status
7	role_type	VARCHAR(20)	NOT NULL	User role (Admin, Staff, User)
8	profile_picture	VARCHAR(255)	NULL	Profile image path
9	created_at	DATETIME	NOT NULL	Record creation time
10	updated_at	DATETIME	NULL	Last update time

2.2.2. EndUser

Sr. No	Name	Data Type	Constraint	Description
1	user_id	INT	PK, FK(User)	Reference to user
2	department_id	INT	FK	Department reference
3	user_type	VARCHAR(50)	NOT NULL	Student/staff type
4	university_id	VARCHAR(50)	UNIQUE	University ID
5	created_at	DATETIME	NOT NULL	Created time
6	updated_at	DATETIME	NULL	Updated time

2.2.3. MaintenanceStaff

Sr. No	Name	Data Type	Constraint	Description
1	user_id	INT	PK, FK(User)	Staff user reference
2	staff_code	VARCHAR(50)	UNIQUE	Unique staff code
3	availability_status	VARCHAR(50)	NOT NULL	Available/Busy
4	workload_count	INT	DEFAULT 0	Assigned tasks count
5	created_at	DATETIME	NOT NULL	Created time
6	updated_at	DATETIME	NULL	Updated time

2.2.4. Admin

Sr. No	Name	Data Type	Constraint	Description
1	user_id	INT	PK, FK(User)	Admin reference

Sr. No	Name	Data Type	Constraint	Description
2	access_level	VARCHAR(20)	NOT NULL	Admin permission level
3	designation	VARCHAR(20)	NOT NULL	Job title
4	created_at	DATETIME	NOT NULL	Created time
5	updated_at	DATETIME	NULL	Updated time

2.2.5. Department

Sr. No	Name	Data Type	Constraint	Description
1	department_id	INT	PK, AUTO_INCREMENT	Unique department ID
2	department_name	VARCHAR(100)	NOT NULL	Department name
3	faculty_name	VARCHAR(100)	NULL	Faculty name
4	created_at	DATETIME	NOT NULL	Created time
5	updated_at	DATETIME	NULL	Updated time

2.2.6. Category

Sr. No	Name	Data Type	Constraint	Description
1	category_id	INT	PK, AUTO_INCREMENT	Unique category ID
2	category_name	VARCHAR(100)	NOT NULL	Complaint type
3	description	VARCHAR(255)	NULL	Category details
4	created_at	DATETIME	NOT NULL	Created time
5	updated_at	DATETIME	NULL	Updated time

2.2.7. PriorityLevel

Sr. No	Name	Data Type	Constraint	Description
1	priority_id	INT	PK, AUTO_INCREMENT	Priority ID
2	priority_name	VARCHAR(50)	NOT NULL	Low/Medium/High
3	description	VARCHAR(255)	NULL	Priority details
4	response_time_hours	INT	NOT NULL	Response time
5	hours_resolution	INT	NOT NULL	Resolution time
6	created_at	DATETIME	NOT NULL	Created time

2.2.8. ComplaintStatus

Sr. No	Name	Data Type	Constraint	Description
1	status_id	INT	PK, AUTO_INCREMENT	Status ID
2	status_name	VARCHAR(50)	NOT NULL	Pending/Resolved
3	description	VARCHAR(255)	NULL	Status details

Sr. No	Name	Data Type	Constraint	Description
4	created_at	DATETIME	NOT NULL	Created time
5	updated_at	DATETIME	NULL	Updated time

2.2.9. Building

Sr. No	Name	Data Type	Constraint	Description
1	building_id	INT	PK, AUTO_INCREMENT	Building ID
2	building_name	VARCHAR(50)	NOT NULL	Building name
3	campus_area	VARCHAR(100)	NULL	Campus area
4	floor	INT	NOT NULL	Number of floors
5	created_at	DATETIME	NOT NULL	Created time
6	updated_at	DATETIME	NULL	Updated time

2.2.10. Location

Sr. No	Name	Data Type	Constraint	Description
1	location_id	INT	PK, AUTO_INCREMENT	Location ID
2	building_id	INT	FK	Building reference
3	room_no	VARCHAR(50)	NOT NULL	Room number
4	floor_no	INT	NOT NULL	Floor number
5	created_at	DATETIME	NOT NULL	Created time
6	updated_at	DATETIME	NULL	Updated time

2.2.11. Complaint

Sr. No	Name	Data Type	Constraint	Description
1	complaint_id	INT	PK, AUTO_INCREMENT	Complaint ID
2	ticket_id	VARCHAR(50)	UNIQUE	Tracking ID
3	title	VARCHAR(150)	NOT NULL	Complaint title
4	description	TEXT	NOT NULL	Complaint details
5	expected_resolution_date	DATE	NULL	Expected fix date
6	submitted_by	INT	FK(User)	User reference
7	category_id	INT	FK	Category reference
8	status_id	INT	FK	Status reference
9	priority_id	INT	FK	Priority reference
10	location_id	INT	FK	Location reference
11	submitted_at	DATETIME	NOT NULL	Submission time
12	updated_at	DATETIME	NULL	Updated time

2.2.12. ComplaintAssignment

Sr. No	Name	Data Type	Constraint	Description
1	assignment_id	INT	PK, AUTO_INCREMENT	Assignment ID
2	complaint_id	INT	FK, UNIQUE	Complaint reference
3	assigned_to	INT	FK	Staff assigned
4	assigned_by	INT	FK	Admin
5	assigned_at	DATETIME	NOT NULL	Assignment time

2.2.13. WorkLog

Sr. No	Name	Data Type	Constraint	Description
1	worklog_id	INT	PK, AUTO_INCREMENT	Work log ID
2	complaint_id	INT	FK	Complaint reference
3	staff_id	INT	FK	Staff reference
4	work_note	TEXT	NOT NULL	Work details
5	logged_at	DATETIME	NOT NULL	Log time

2.2.14. Resolution

Sr. No	Name	Data Type	Constraint	Description
1	resolution_id	INT	PK, AUTO_INCREMENT	Resolution ID
2	complaint_id	INT	FK, UNIQUE	Complaint reference
3	resolved_by	INT	FK	Resolver
4	resolved_at	DATETIME	NULL	Resolution time
5	resolution_summary	TEXT	NOT NULL	Final solution
6	resolution_date	DATE	NOT NULL	Date of resolution

2.2.15. ComplaintImage

Sr. No	Name	Data Type	Constraint	Description
1	image_id	INT	PK, AUTO_INCREMENT	Image ID
2	complaint_id	INT	FK	Complaint reference
3	attachment_type_id	INT	FK	Attachment type
4	uploaded_by	INT	FK	Uploader
5	image_url	VARCHAR(255)	NOT NULL	Image path
6	uploaded_at	DATETIME	NOT NULL	Upload time

2.2.16. Comment

Sr. No	Name	Data Type	Constraint	Description
1	comment_id	INT	PK, AUTO_INCREMENT	Comment ID

Sr. No	Name	Data Type	Constraint	Description
2	complaint_id	INT	FK	Complaint reference
3	user_id	INT	FK	User reference
4	content	TEXT	NOT NULL	Comment text
5	comment_type	VARCHAR(50)	NULL	Type of comment
6	comment_at	DATETIME	NOT NULL	Comment time

2.2.17. Notification

Sr. No	Name	Data Type	Constraint	Description
1	notification_id	INT	PK, AUTO_INCREMENT	Notification ID
2	user_id	INT	FK	Recipient
3	complaint_id	INT	FK, NOT NULL	Related complaint
4	template_id	INT	FK	Template reference
5	message	TEXT	NOT NULL	Notification message
6	is_read	BOOLEAN	DEFAULT 0	Read status
7	created_at	DATETIME	NOT NULL	Created time

2.2.18. StatusHistory

Sr. No	Name	Data Type	Constraint	Description
1	history_id	INT	PK, AUTO_INCREMENT	History ID
2	complaint_id	INT	FK	Complaint reference
3	old_status_id	INT	FK	Previous status
4	new_status_id	INT	FK	Updated status
5	changed_by	INT	FK	User who changed
6	remarks	TEXT	NULL	Change reason
7	changed_at	DATETIME	NOT NULL	Change time

2.2.19. Feedback

Sr. No	Name	Data Type	Constraint	Description
1	feedback_id	INT	PK, AUTO_INCREMENT	Feedback ID
2	complaint_id	INT	FK, UNIQUE	Complaint reference
3	user_id	INT	FK	User reference
4	rating	INT	CHECK (1–5)	Rating score
5	feedback_text	TEXT	NULL	Feedback
6	submitted_at	DATETIME	NOT NULL	Submission time

2.2.20. AuditLog

Sr. No	Name	Data Type	Constraint	Description
1	audit_id	INT	PK, AUTO_INCREMENT	Audit ID
2	user_id	INT	FK, NULL	User reference
3	action_type	VARCHAR(100)	NOT NULL	Action performed
4	affected_table	VARCHAR(100)	NOT NULL	Table affected
5	action_time	DATETIME	NOT NULL	Action time
6	details	TEXT	NULL	Extra details

2.2.21. StaffSpecialization

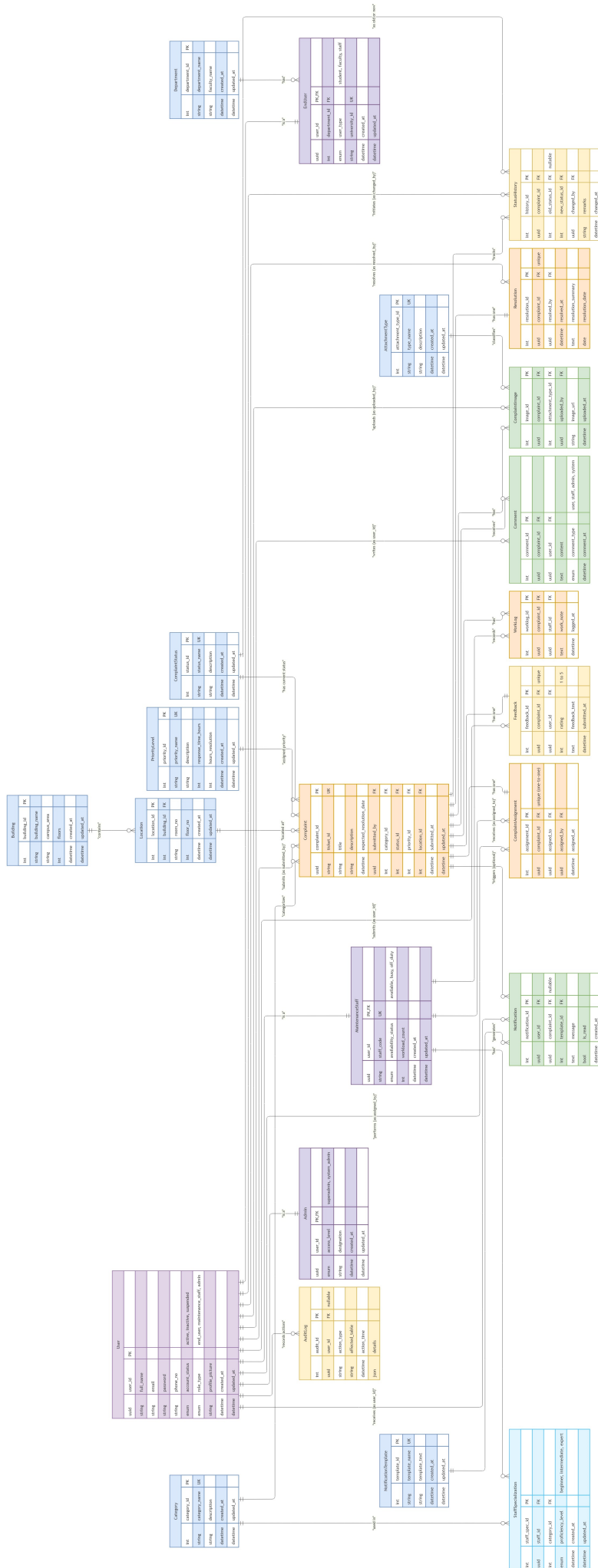
Sr. No	Name	Data Type	Constraint	Description
1	staff_spec_id	INT	PK, AUTO_INCREMENT	ID
2	staff_id	INT	FK	Staff reference
3	category_id	INT	FK	Category reference
4	proficiency_level	VARCHAR(50)	NOT NULL	Skill level
5	created_at	DATETIME	NOT NULL	Created time
6	updated_at	DATETIME	NULL	Updated time

2.3. BUSINESS RULES

1. Every user must be registered and authenticated before accessing the system.
2. Each user shall have a single role (End User, Maintenance Staff, or Administrator) that determines their permissions.
3. An End User may submit multiple maintenance requests, but each request must be associated with exactly one user.
4. Every maintenance request must belong to a specific category and maintain one active status at a time.
5. Only Administrators can assign or reassign maintenance requests to Maintenance Staff.
6. A Maintenance Staff member may handle multiple maintenance requests simultaneously, while each request can be assigned to only one staff member at a time.
7. The status of a maintenance request must follow the predefined workflow from submission to completion.
8. Feedback can only be submitted by the requester after the maintenance request has been completed.
9. The system must maintain a history of request updates, assignments, and status changes for auditing purposes.

2.4. ENTITY RELATIONSHIP DIAGRAM

Complaint Management System



LOGICAL DATABASE DESIGN

3.1. RELATIONAL SCHEMA

The ERD of the NAMAL Complaint Management System was transformed into a relational schema by converting each entity into a table and its attributes into columns. Primary keys were defined to uniquely identify records, while relationships between entities were implemented using foreign keys. Many-to-many relationships, where applicable, were resolved through associative tables containing the primary keys of the related entities. This transformation ensures data integrity, minimizes redundancy, and provides a structured database design suitable for implementation in any relational DBMS.

3.2. FUNCTIONAL DEPENDENCIES

- $user_id \rightarrow account_status, role_type, created_at, updated_at, full_name, email, password, phone_no, profile_picture$
- $user_id \rightarrow department_id, user_type, university_id$
- $user_id \rightarrow access_level, designation$
- $user_id \rightarrow availability_status, workload_count, staff_code$
- $staff_spec_id \rightarrow staff_id, category_id, proficiency_level, created_at, updated_at$
- $complaint_id \rightarrow description, expected_resolution_at, submitted_by, category_id, status_id, priority_id, location_id, submitted_at, updated_at, ticket_id, title$
- $ticket_id \rightarrow complaint_id$
- $assignment_id \rightarrow complaint_id, assigned_to, assigned_by, assigned_at$
- $complaint_id \rightarrow assignment_id$
- $comment_id \rightarrow complaint_id, user_id, content, comment_type, comment_at$
- $resolution_id \rightarrow complaint_id, resolved_by, resolution_details, resolution_summary, resolution_date$
- $complaint_id \rightarrow resolution_id$
- $feedback_id \rightarrow complaint_id, user_id, rating, feedback_text, submitted_at$
- $complaint_id \rightarrow feedback_id$
- $audit_id \rightarrow user_id, action_time, details, action_type, affected_table, remarks$
- $history_id \rightarrow complaint_id, old_status_id, new_status_id, changed_by, changed_at, remarks$
- $notification_id \rightarrow user_id, complaint_id, template_id, message, is_read, created_at$
- $template_id \rightarrow template_text, created_at, updated_at, template_name$

- `template_name` → `template_id`
- `image_id` → `complaint_id`, `attachment_type_id`, `uploaded_by`, `uploaded_at`, `image_url`
- `attachment_type_id` → `description`, `created_at`, `updated_at`, `type_name`
- `type_name` → `attachment_type_id`
- `category_id` → `description`, `created_at`, `updated_at`, `category_name`
- `category_name` → `category_id`
- `priority_id` → `description`, `response_time_hours`, `hours_resolution`, `created_at`, `updated_at`, `priority_name`
- `priority_name` → `priority_id`
- `status_id` → `description`, `created_at`, `updated_at`, `status_name`
- `status_name` → `status_id`
- `department_id` → `created_at`, `updated_at`, `department_name`, `faculty_name`
- `department_name` → `department_id`
- `location_id` → `building_id`, `floor_no`, `created_at`, `updated_at`, `room_no`
- `building_id` → `floors`, `created_at`, `updated_at`, `building_name`, `campus_area`

3.3. NORMALIZATION

3.3.1. Partial Dependencies

Since all tables use a single-attribute primary key, there are no partial functional dependencies. Therefore, all relations satisfy Second Normal Form (2NF).

3.3.2. Transitive Dependencies

A transitive dependency is a type of functional dependency that occurs in a database when a non-key attribute depends on another non-key attribute, rather than depending directly on the primary key.

More formally, if we have three attributes A, B, and C such that:

- $A \rightarrow B$ (A determines B), and
- $B \rightarrow C$ (B determines C)

1st NF

- `user_id` → `account_status`, `role_type`, `created_at`, `updated_at`, `full_name`, `email`, `password`, `phone_no`, `profile_picture`
- `user_id` → `department_id`, `user_type`, `university_id`
- `user_id` → `access_level`, `designation`
- `user_id` → `availability_status`, `workload_count`, `staff_code`

- staff_spec_id → staff_id, category_id, proficiency_level, created_at, updated_at
- complaint_id → description, expected_resolution_at, submitted_by, category_id, status_id, priority_id, location_id, submitted_at, updated_at, ticket_id, title
- ticket_id → complaint_id
- assignment_id → complaint_id, assigned_to, assigned_by, assigned_at
- complaint_id → assignment_id
- comment_id → complaint_id, user_id, content, comment_type, comment_at
- resolution_id → complaint_id, resolved_by, resolution_details, resolution_summary, resolution_date
- complaint_id → resolution_id
- feedback_id → complaint_id, user_id, rating, feedback_text, submitted_at
- complaint_id → feedback_id
- audit_id → user_id, action_time, details, action_type, affected_table, remarks
- history_id → complaint_id, old_status_id, new_status_id, changed_by, changed_at, remarks
- notification_id → user_id, complaint_id, template_id, message, is_read, created_at
- template_id → template_text, created_at, updated_at, template_name
- template_name → template_id
- image_id → complaint_id, attachment_type_id, uploaded_by, uploaded_at, image_url
- attachment_type_id → description, created_at, updated_at, type_name
- type_name → attachment_type_id
- category_id → description, created_at, updated_at, category_name
- category_name → category_id
- priority_id → description, response_time_hours, hours_resolution, created_at, updated_at, priority_name
- priority_name → priority_id
- status_id → description, created_at, updated_at, status_name
- status_name → status_id
- department_id → created_at, updated_at, department_name, faculty_name
- department_name → department_id
- location_id → building_id, floor_no, created_at, updated_at, room_no
- building_id → floors, created_at, updated_at, building_name, campus_area

The given functional dependencies satisfy First Normal Form (1NF) as all attributes are atomic and no repeating groups exist in any relation.

2nd NF

In 2NF, the main requirement is that the schema must first be in First Normal Form (1NF), and then all non-key attributes must be fully functionally dependent on the whole primary key, meaning there should be no partial dependency.

The following functional dependencies confirm the structure in 2NF:

- user_id → account_status, role_type, created_at, updated_at, full_name, email, password, phone_no, profile_picture
- user_id → department_id, user_type, university_id
- user_id → access_level, designation
- user_id → availability_status, workload_count, staff_code
- staff_spec_id → staff_id, category_id, proficiency_level, created_at, updated_at
- complaint_id → description, expected_resolution_at, submitted_by, category_id, status_id, priority_id, location_id, submitted_at, updated_at, ticket_id, title
- assignment_id → complaint_id, assigned_to, assigned_by, assigned_at
- comment_id → complaint_id, user_id, content, comment_type, comment_at
- resolution_id → complaint_id, resolved_by, resolution_details, resolution_summary, resolution_date
- feedback_id → complaint_id, user_id, rating, feedback_text, submitted_at
- audit_id → user_id, action_time, details, action_type, affected_table, remarks
- history_id → complaint_id, old_status_id, new_status_id, changed_by, changed_at, remarks
- notification_id → user_id, complaint_id, template_id, message, is_read, created_at
- image_id → complaint_id, attachment_type_id, uploaded_by, uploaded_at, image_url
- category_id → description, created_at, updated_at, category_name
- priority_id → description, response_time_hours, hours_resolution, created_at, updated_at, priority_name
- status_id → description, created_at, updated_at, status_name
- department_id → created_at, updated_at, department_name, faculty_name
- location_id → building_id, floor_no, created_at, updated_at, room_no
- building_id → floors, created_at, updated_at, building_name, campus_area

Since all attributes are fully dependent on their respective primary keys and no partial dependency exists, the schema satisfies the requirements of Second Normal Form (2NF).

3rd NF

To achieve Third Normal Form (3NF) in the database, the schema was further refined after ensuring compliance with Second Normal Form (2NF). The main requirement of 3NF is that

there should be no transitive dependency, meaning that non-key attributes must depend only on the primary key and not on other non-key attributes.

- `user_id` → `account_status`, `role_type`, `full_name`, `email`, `password`, `phone_no`, `profile_picture`, `created_at`, `updated_at`
- `department_id` → `department_name`, `faculty_name`
- `staff_id` → `user_id`, `staff_code`, `availability_status`, `workload_count`, `designation`
- `complaint_id` → `title`, `description`, `submitted_by`, `category_id`, `status_id`, `priority_id`, `location_id`, `submitted_at`, `updated_at`
- `status_id` → `status_name`, `description`
- `priority_id` → `priority_name`, `response_time_hours`, `hours_resolution`
- `category_id` → `category_name`, `description`
- `location_id` → `building_id`, `floor_no`, `room_no`
- `building_id` → `building_name`, `floors`, `campus_area`
- `assignment_id` → `complaint_id`, `assigned_to`, `assigned_by`, `assigned_at`
- `feedback_id` → `complaint_id`, `user_id`, `rating`, `feedback_text`, `submitted_at`
- `comment_id` → `complaint_id`, `user_id`, `content`, `comment_type`, `comment_at`
- `resolution_id` → `complaint_id`, `resolved_by`, `resolution_details`, `resolution_summary`, `resolution_date`
- `notification_id` → `user_id`, `complaint_id`, `template_id`, `message`, `is_read`, `created_at`
- `template_id` → `template_name`, `template_text`, `created_at`, `updated_at`
- `audit_id` → `user_id`, `action_type`, `action_time`, `affected_table`, `details`, `remarks`
- `history_id` → `complaint_id`, `old_status_id`, `new_status_id`, `changed_by`, `changed_at`, `remarks`
- `image_id` → `complaint_id`, `attachment_type_id`, `uploaded_by`, `uploaded_at`, `image_url`
- `attachment_type_id` → `type_name`, `description`

After decomposition, all non-key attributes depend only on their respective primary keys, and no transitive dependency remains. Therefore, the schema satisfies the requirements of Third Normal Form (3NF).

IMPLEMENTATION

4.1. LANGUAGE/Framework

Technology Selection:

React: Used to develop a responsive, interactive, and user-friendly graphical interface.

Node.js: Used as the server-side runtime environment to process client requests efficiently.

Express.js: Used to build APIs, manage routing, and handle backend operations.

MySQL: Used as the relational database management system for secure and efficient storage of complaint-related data.

Reason for Selection: This technology stack provides scalability, maintainability, high performance, and seamless integration between the frontend, backend, and database.

4.2. DATABASE CONNECTIVITY

The application uses React as the frontend (GUI), Node.js with Express.js as the backend, and MySQL as the database.

React communicates with the backend through RESTful APIs using HTTP requests (Fetch/Axios). The backend built in Node.js and Express handles these requests and performs all business logic. For database connectivity, the backend uses the `mysql2` driver to connect with MySQL using standard configuration parameters such as host (localhost), username, password, and database name (port 3306).

The system follows a client-server architecture, where React acts as the client, Express/Node.js serves as the application server, and MySQL is used for persistent data storage. This ensures efficient and structured communication between the user interface and the database.

4.3. STORED PROCEDURES AND FUNCTIONS

Sr.	Object Name	Object Type	Description	Input Parameters	Output / Return Value
1	sp_CreateRequest	Stored Procedure	Insert a new maintenance request into the database.	user_id, category_id, description	Success / Failure message
2	sp_AssignRequest	Stored Procedure	Assign a maintenance request to a staff member.	user_id, staff_id	Assignment confirmation

Sr.	Object Name	Object Type	Description	Input Parameters	Output / Return Value
3	fn_TotalRequests	Function	Calculate the total number of maintenance requests.	user_id (optional)	Integer (total count)
4	fn_PendingRequests	Function	Return the count of pending maintenance requests.	user_id (optional)	Integer (pending count)
5	trg_RequestStatusUpdate	Trigger	Automatically create a log entry when a request status is changed.	UPDATE on maintenance_request table	Log entry in audit table
6	trg_RequestInsert	Trigger	Automatically set the timestamp when a new request is inserted.	INSERT on maintenance_request	Auto created_at value
7	vw_AllRequests	View	Display a combined view of all maintenance requests.	None	List of all requests
8	vw_PendingRequests	View	Display only pending maintenance requests.	None	Filtered pending request list

4.4. INTERFACES

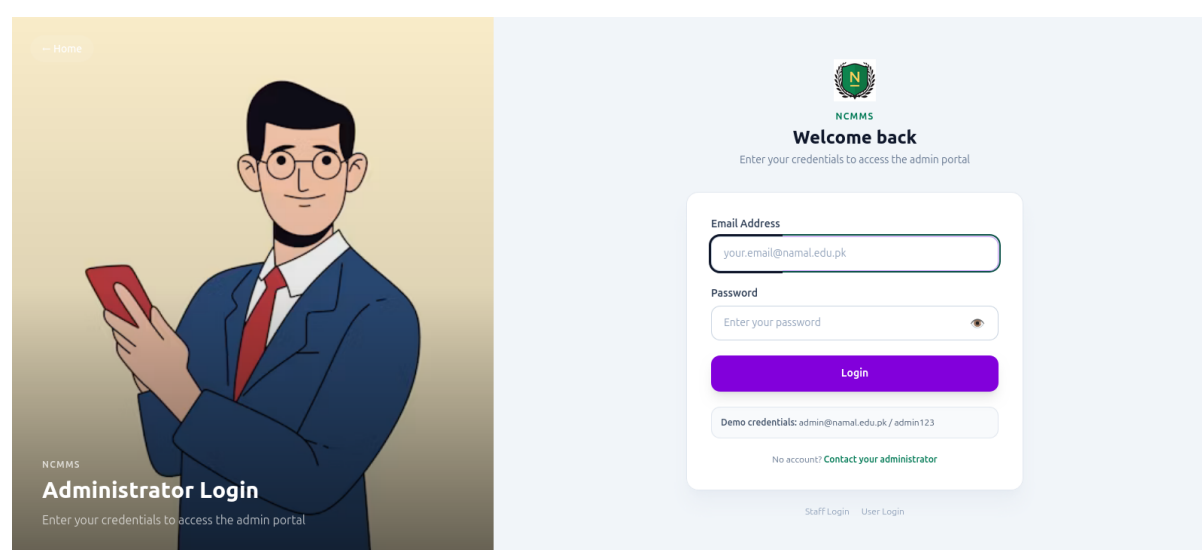


Figure 4.1: Administrator Login Page

Description: This page allows administrators to securely log in to the Complaint Management System using their email address and password.

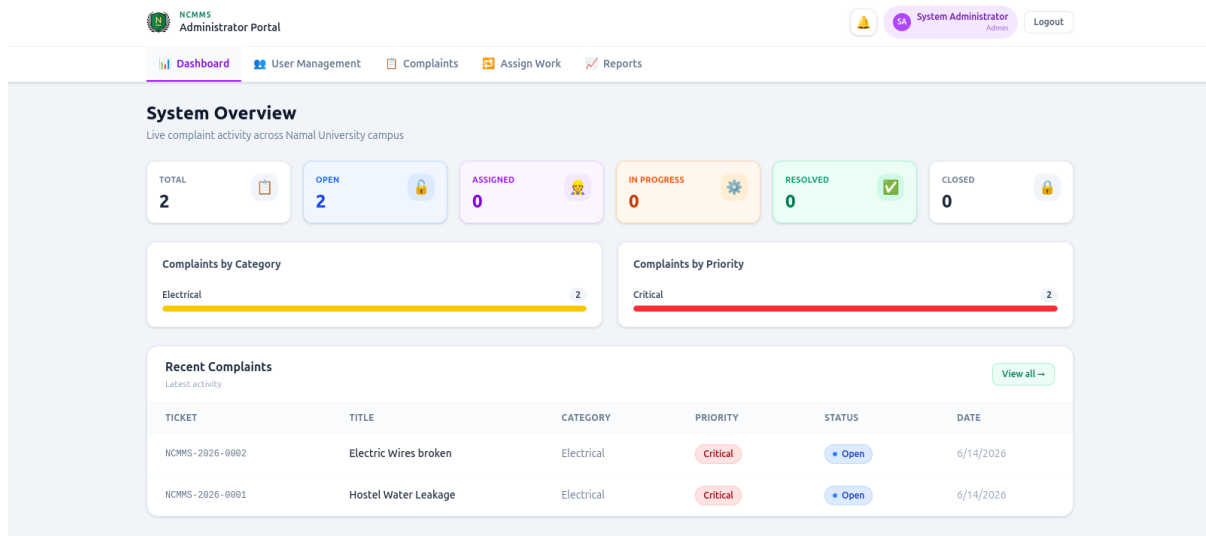


Figure 4.2: Admin Dashboard

Description: This is an admin dashboard of a complaint management system showing overall complaint status, analytics, and recent complaint records.

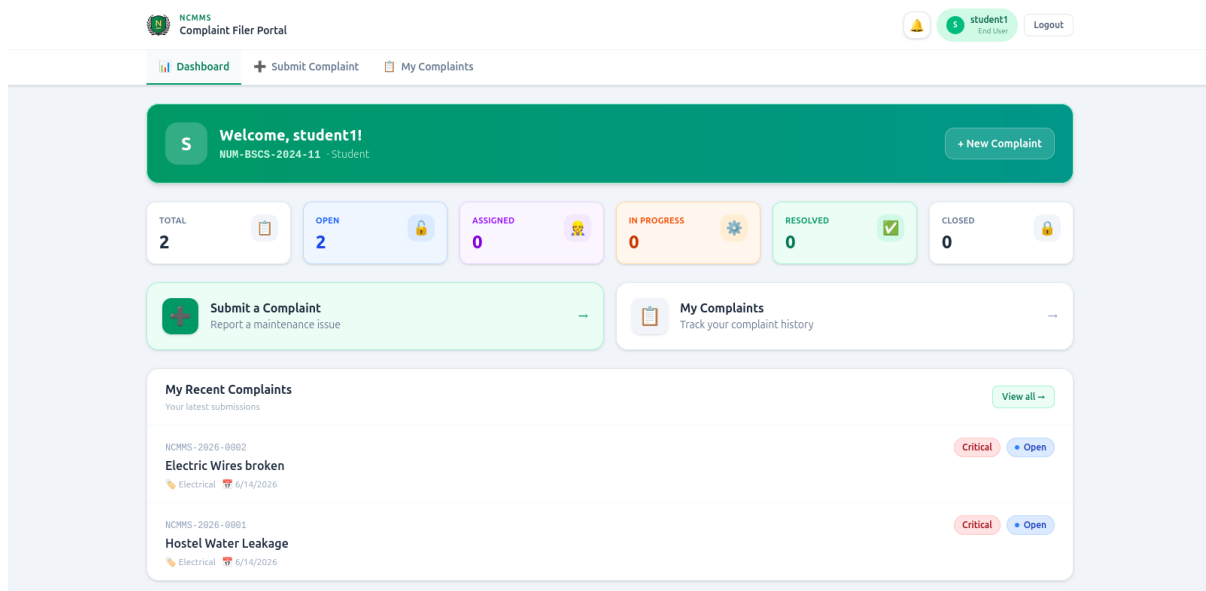


Figure 4.3: Complaint Filer Portal

Description: This is a NCMMS Complaint Filer Portal dashboard where students can submit and track their maintenance complaints.

NCMMS User Management

System Administrator Admin Logout

Dashboard User Management Complaints Assign Work Reports

All Users **Create Complaint Filer** Create Maintenance Staff

Create Complaint Filer Account

For students, faculty, or university staff who will submit complaints.

Full Name * Email *

Temporary Password * Phone

User Type * University ID *

Department Format: NUM-BSCS-2024-75

Create Account

Figure 4.4: Create Complaint Filer Account Page

Description: Page where the System Administrator can register new students, faculty, or staff by filling in their name, email, password, user type, university ID, and department.

NCMMS Maintenance Staff Portal

staff11 Maintenance Staff Logout

Dashboard My Tasks Work Report

Welcome, staff11
STAFF-123 available 2 Active tasks

TOTAL 2 OPEN 0 ASSIGNED 2 IN PROGRESS 0 RESOLVED 0 WORKLOAD 2

View My Tasks
See all assigned complaints

Submit Work Report
Log progress and update status

Active Tasks

Assigned & In progress View all →

NCMMS-2026-0002
Electric Wires broken
Hostel - Boys — B-104 Electrical Critical Assigned

NCMMS-2026-0001
Hostel Water Leakage
Academic Block A — 201 Electrical Critical Assigned

Figure 4.5: Maintenance Staff Dashboard

Description: This is the Maintenance Staff Dashboard where staff can view their assigned tasks and submit work reports to update complaint progress.

REFERENCES

1. React Official Documentation, “Getting Started with React,” 2026. [Online]. Available: <https://react.dev>
2. Node.js Documentation, “Node.js Guide,” 2026. [Online]. Available: <https://nodejs.org>
3. MySQL Documentation, “MySQL 8.0 Reference Manual,” 2026. [Online]. Available: <https://dev.mysql.com>
4. Express.js Guide, “Express Framework Documentation,” 2026. [Online]. Available: <https://expressjs.com>

APPENDIX: AI USAGE AND PROMPTS

1. User & Student Personas (Filing Complaints)

These prompts show how a regular user interacts with the React frontend to lodge an issue.

- *Prompt:* “As a student living in the hostel, how do I submit a high-priority complaint about the Wi-Fi being down in Block B, and how can I attach a screenshot of the speed test?”
- *Prompt:* “Draft a user flow for a faculty member who needs to report a broken projector in Lecture Hall 3, ensuring they can select the correct department (IT Support) from a dropdown.”
- *Prompt:* “How does a student track the real-time status of their submitted complaint (e.g., ‘Pending’ to ‘In Progress’ to ‘Resolved’) from their personal dashboard?”

2. Admin & Faculty Personas (Managing Complaints)

These prompts highlight the backend logic (Node.js/Express) and database updates (MySQL) when an admin manages the incoming queue.

- *Prompt:* “As the Admin of the Namal Maintenance Department, how do I view a filtered list of all ‘Unassigned’ complaints related to ‘Electrical Works’?”
- *Prompt:* “Show the process of an admin assigning a specific complaint to a technician and adding an internal note that updates the database status.”
- *Prompt:* “How does the system generate a monthly summary report showing the average resolution time for complaints across different categories?”

3. System Architecture & Database (Backend Focus)

Great for the technical implementation section of your report. These focus on how Express handles requests and how MySQL stores them.

- *Prompt:* “Design a MySQL schema for the complaints table that links to a user’s table via a foreign key (user_id), including fields for timestamps, category, priority, and status.”
- *Prompt:* “Write an Express.js POST API endpoint (/api/complaints) that validates user input, handles database errors, and returns a successful JSON response with the new complaint ID.”
- *Prompt:* “How does the React frontend securely send the JWT token in the authorization header when fetching user-specific complaints from the protected Node.js route?”

4. Edge Cases & Testing Prompts

Including these in your report proves that you didn’t just build a “happy path” system, but actually thought about security and errors.

- *Prompt:* “What happens when a user tries to submit an empty complaint form? How does the React state handle and display frontend validation errors?”

- *Prompt:* “Simulate a scenario where two admins try to update the status of the same complaint simultaneously. How does the backend prevent data conflicts?”
- *Prompt:* “Test the system’s security: If a student tries to manually alter the API URL parameter to view another student’s complaint ID, how does the Node.js middleware block the unauthorized request?”